

Python을 이용한 파일 처리

03 여러 파일 다루기

목차 03 여러 파일 다루기

1. 경로 기초
2. Path 객체와 기본 메서드
3. Path 객체의 속성
4. 파일 변경하기

1. 경로 기초

- 디렉터리

- 폴더를 뜻하는 다른 말, 프로그래밍 및 명령줄에서는 일반적으로 이 표현을 사용

- 루트 디렉터리

- 최상위 디렉터리
- Windows의 경우 일반적으로 "C:\", macOS의 경우 '/'

- 절대 경로

- 루트 디렉터리부터 나열한 경로
- "C:\Users\Name\file.txt" 등

- 상대 경로

- 작업 디렉터리를 기준으로 나열한 경로
- 현재 작업 디렉터리가 "C:\Users"일 경우 "Name\file.txt" 등



• 경로 구분자

- 디렉터리와 디렉터리 및 파일을 구분하는 단위
- Windows에서는 '\'를, macOS에서는 '/'를 사용
 - '\'는 시스템 설정에 따라 '¥' 혹은 '₩'으로 나타날 수 있음

• 작업 디렉터리

- 셸(cmd, PowerShell, zsh 등) 혹은 프로그램이 현재 가리키는 디렉터리
- '.'으로 표현



- 부모 디렉터리
 - 작업 디렉터리의 상위에 있는 디렉터리
 - `".."`으로 표현
- 홈 디렉터리
 - 사용자의 기본 디렉터리, Windows에서는 일반적으로 `"C:\Users\{사용자명}"`, macOS에서는 일반적으로 `"/Users/{사용자명}"`을 가리킴
 - `'~'`로 표현

2. Path 객체와 기본 메서드

- Python에서 파일 경로를 다루는 라이브러리
- 운영 체제에 종속적이지 않고 독립적으로 동작
 - `'/'`, `'\\'` 등을 신경 쓰지 않아도 됨(프로그램 작성시 호환성을 위해 `'/'` 사용)
 - 호환성이 높은 프로그램을 제작할 수 있음
 - 단일 운영 체제에 대해 종속적인 작업이 필요한 경우 `os`, `os.path` 라이브러리 사용
- 경로를 문자열이 아닌 객체로 취급
 - 단순히 문자열로 다루는 것보다 더 안정적인 작업 가능



```
from pathlib import Path

# 현재 경로

# p = Path(".")와 동일
p = Path()

# 현재 경로에 있는 "main.py" 파일의 내용을 읽어 출력

# 상대 경로이므로 (p / "main") 대신
# Path("main.py")를 사용할 수도 있음
print((p / "main.py").read_text("utf-8"))
```

- pathlib에서 경로를 표현하는 기본 객체
 - 파일 시스템에 따라 WindowsPath와 PosixPath 중 하나로 만들어짐
- 경로에 대한 다양한 속성과 메서드 및 연산 제공
- pathlib의 사실상 유일한 사용 요소이므로 일반적으로 **"from pathlib import Path"** 형태로 임포트하여 사용



- **cwd**: 작업 디렉터리 객체를 절대 경로로 반환, 클래스 메서드
- **home**: 홈 디렉터리 객체를 절대 경로로 반환, 클래스 메서드
- **exists**: 경로가 실제로 존재하는지 확인
- **is_dir**: 경로가 가리키는 것이 디렉터리인지 확인
- **is_file**: 경로가 가리키는 것이 일반 파일인지 확인

```
from pathlib import Path

print(Path.cwd())
# C:\Users\Name\current_working_directory

# 동작은 하지만 다음과 같이 사용하지 않음
# p = Path()
# print(p.cwd())
```

- **cwd**: 작업 디렉터리 객체를 반환
- 클래스 메서드이므로 할당해서 사용하지 않고 Path 자체에서 직접 호출

```
from pathlib import Path

print(Path.home())
# C:\Users\Name

# 동작은 하지만 다음과 같이 사용하지 않음
# p = Path()
# print(p.home())
```

- **home**: 홈 디렉터리 객체를 반환
- 클래스 메서드이므로 할당해서 사용하지 않고 Path 자체에서 직접 호출



exist, is_dir, is_file

Python을 이용한 파일
처리

03 여러 파일 다루기

2. Path 객체와 기본
메서드

```
from pathlib import Path

p = Path.cwd()

print(p.exists())
# True

p = Path("nonexisting_path")

print(p.exists())
# False
```

- **exists**: 해당 경로가 존재하는지 확인

```
from pathlib import Path

p = Path.cwd() # 항상 디렉터리

print(p.is_dir())
# True

p /= "testfile.txt"
print(p.is_dir())
# False

p = Path("nonexisting_path")
print(p.is_dir())
# False
```

- **is_dir**: 해당 경로가 디렉터리인지 확인
- 만약 존재하지 않는 경로라면 False 반환

```
from pathlib import Path

p = Path.cwd() / "testfile.txt"

print(p.is_file())
# True

p = Path.cwd()
print(p.is_file())
# False

p = Path("nonexisting_path")
print(p.is_file())
# False
```

- **is_file**: 해당 경로가 파일인지 확인
- 만약 존재하지 않는 경로라면 False 반환



```
from pathlib import Path

# 작업 디렉터리: C:\Users\Name
p = Path.cwd() / "dir/another_dir" / "more_dir/file.txt"
print(p)

# C:\Users\Name\dir\another_dir\more_dir\file.txt
```

Path 객체는 `'/'` 연산자를 경로 연결하기로 사용할 수 있음



- **expanduser**: 홈 디렉터리 표기(~)를 사용한 경로 표현을 절대 경로 표현으로 반환
- **resolve**: ".." 등을 사용한 경로 표현을 절대 경로 표현으로 반환
- **absolute**: 일반적인 상대 경로를 절대 경로 표현으로 반환

- **iterdir**: 경로 안의 모든 파일 탐색
- **walk**: 경로 안의 모든 파일 탐색, 재귀적



expanduser, resolve, absolute

Python을 이용한 파일
처리

03 여러 파일 다루기

2. Path 객체와 기본
메서드

```
from pathlib import Path

p = Path("~/dir")
p = p.expanduser()

print(p)
# C:\Users\Name\dir
```

- **expanduser**: '~'를 포함한 경로를 절대 경로로 변경하여 반환
- 존재하지 않는 경로도 반환

```
from pathlib import Path

p = Path("..")
p = p.resolve()

print(p)
# C:\Users\Name

p = Path("nonexisting_path")
p = p.resolve()

print(p)
# C:\Users\Name\dir\nonexisting_path

p = Path("nonexisting_path")
p = p.resolve(True)
# FileNotFoundError
```

- **resolve**: ".." 등의 구성 요소를 분석하여 절대 경로로 반환
- 기본적으로 존재하지 않는 경로도 반환
- **True**를 전달하면 존재하지 않는 경로에 **FileNotFoundError** 발생

```
from pathlib import Path

p = Path()
print(p)
# .

p = p.absolute()
print(p)
# C:\Users\Name\dir

p = Path("..")
p = p.absolute()
print(p)
# C:\Users\Name\dir\..
```

- **absolute**: 경로 기호를 분석하지 않고 절대 경로로 반환
- 존재하지 않는 경로도 반환


```
from pathlib import Path

p = Path.cwd()
if p.is_dir():
    for child in p.iterdir():
        print(child)
        # C:\Users\Name\dir\dir1
        # C:\Users\Name\dir\file1.txt
        # C:\Users\Name\dir\file2.txt
```

- 경로가 디렉터리일 때 해당 디렉터리의 파일을 반환하는 제너레이터를 반환
- 경로가 디렉터리가 아니거나 존재하지 않을 경우 운영체제마다 동작이 다름 → 일관적인 예외 처리 불가능
 - **iterdir** 호출 전 항상 **is_dir** 메서드로 확인을 하는 것이 좋음

- 디렉터리 안에 있는 모든 파일에 대해 재귀적으로 탐색
 - 만약 디렉터리 안에 또다른 디렉터리가 있는 경우 해당 디렉터리 안의 내용도 탐색
- (경로 객체, [경로 안의 디렉터리], [경로 안의 파일]) 형태의 튜플을 가지는 이터레이터를 반환
- `iterdir`과 마찬가지로 `is_dir` 호출 결과를 확인하는 것이 좋음
- Python 3.12에 추가된 기능
 - 현재 사용중인 Python 버전이 3.12보다 낮을 경우, 비슷한 동작을 하는 `os.walk` 함수로 대신할 수 있음
 - 객체의 메서드로 호출하는 대신 함수의 전달 인자로 Path 객체 혹은 경로 문자열 전달
 - 이 경우 (경로 문자열, [경로 안의 디렉터리], [경로 안의 파일]) 형태의 튜플을 가지는 이터레이터를 반환

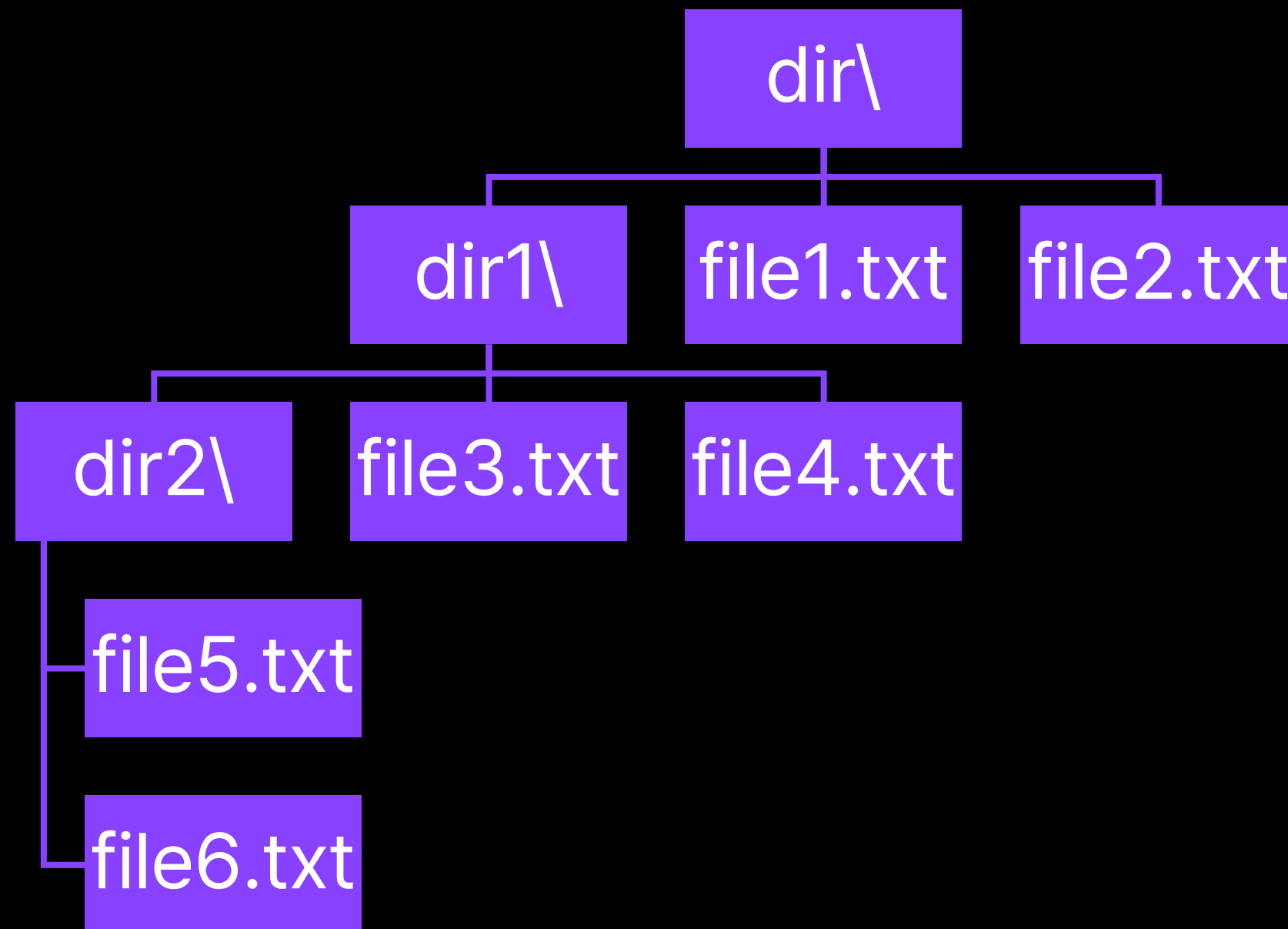


walk 예제

Python을 이용한 파일
처리

03 여러 파일 다루기

2. Path 객체와 기본
메서드



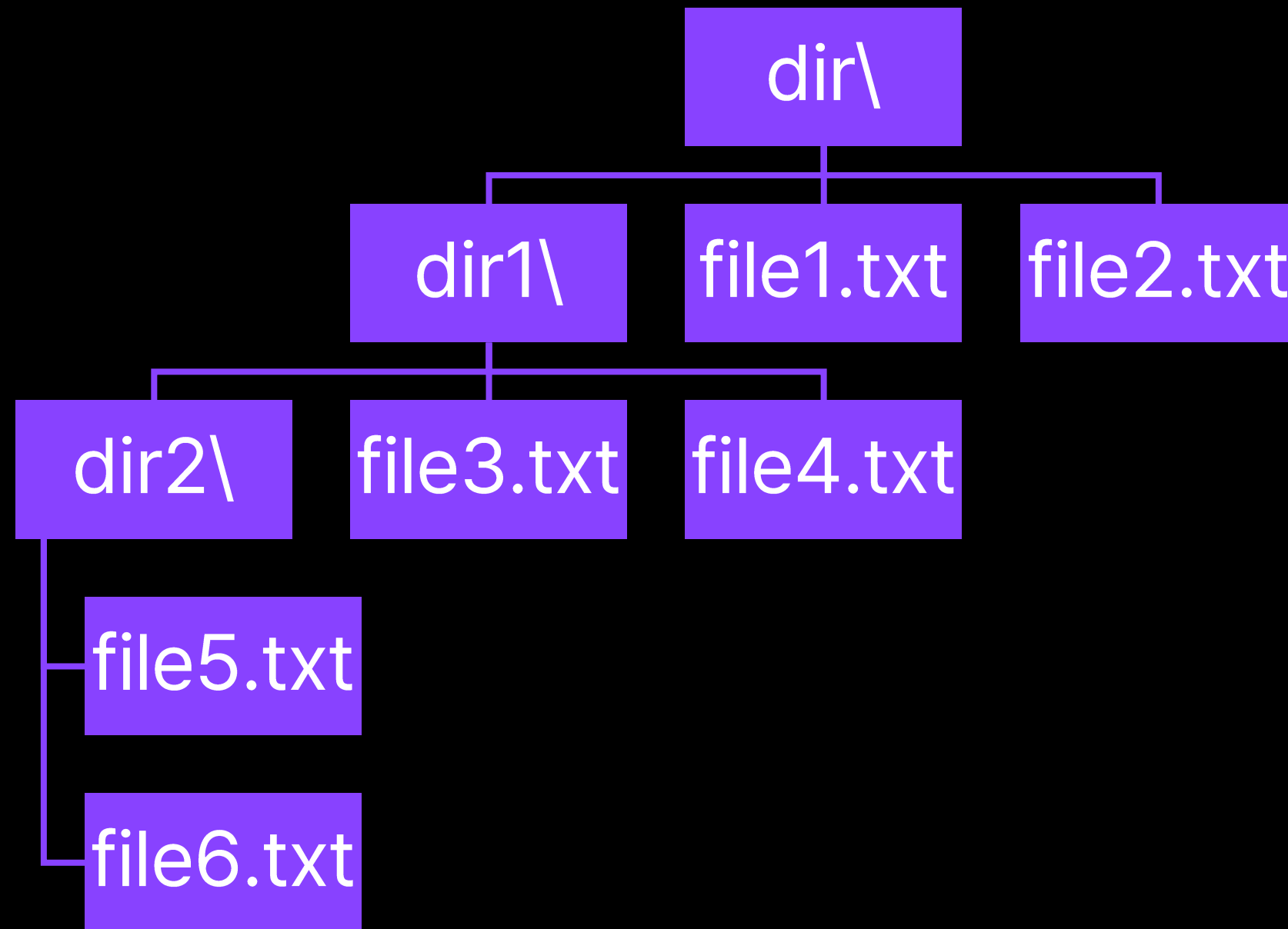
디렉터리 구조

```
from pathlib import Path

p = Path.cwd()

if p.is_dir():
    for root, dirs, files in p.walk():
        print(root, dirs, files)
        # C:\Users\Name\dir ['dir1'] ['file1.txt', 'file2.txt']
        # C:\Users\Name\dir\dir1 ['dir2'] ['file3.txt', 'file4.txt']
        # C:\Users\Name\dir\dir1\dir2 [] ['file5.txt', 'file6.txt']
```

코드 및 결과



디렉터리 구조

```
import os
from pathlib import Path

p = Path.cwd()

if p.is_dir():
    for root, dirs, files in os.walk(p):
        print(root, dirs, files)
        # C:\Users\Name\dir ['dir1'] ['file1.txt', 'file2.txt']
        # C:\Users\Name\dir\dir1 ['dir2'] ['file3.txt', 'file4.txt']
        # C:\Users\Name\dir\dir1\dir2 [] ['file5.txt', 'file6.txt']
```

코드 및 결과

3. Path 객체의 속성

- **parent**: 경로의 1차 상위 항목(Path)
- **parents**: 경로의 상위 항목에 대한 시퀀스
- **name**: 마지막 경로 구성 요소를 나타내는 항목(문자열)
- **suffix**: **name**에서 파일명은 제외하고 '.'을 포함하는 확장자(문자열)
- **stem**: **name**에서 확장자를 제외한 마지막 경로 구성 요소(문자열)

```
from pathlib import Path

# 작업 디렉터리: "C:\Users\Name\dir"
p = Path.cwd()

print(p.parent)
# C:\Users\Name

# 작업 디렉터리: "."
p = Path()

print(p.parent)
# .
```

- 경로의 바로 상위 디렉터를 가리킴(Path 객체)
- 단순한 어휘 분석이므로 절대 경로가 아닌 경우 제대로 된 상위 디렉터를 가리키지 않을 수 있음
 - **resolve** 등의 메서드를 이용하여 절대 경로로 변환해야 항상 바람직한 값을 얻을 수 있음

```
from pathlib import Path

# 작업 디렉터리: "C:\Users\Name\dir"
p = Path().resolve()

for parent in p.parents:
    print(parent)
    # C:\Users\Name
    # C:\Users
    # C:\

print(p.parents[0])
# C:\Users\Name
print(p.parents)
# <WindowsPath.parents>
```

- 경로의 상위 디렉터리를 루트 디렉터리까지의 내용을 가지고 있는 시퀀스
 - 순서는 1차 상위 디렉터리 → 루트 디렉터리 순
 - 내용의 직접 출력은 불가능
- 시퀀스 안의 요소는 Path 객체
- **parent**와 마찬가지로 **resolve** 등의 메서드로 절대 경로로 변환한 후 사용하는 것이 좋음

```
from pathlib import Path

# C:\Users\Name\dir\somedir\file.txt
p = Path.cwd() / "somedir/file.txt"

print(p.name)
# file.txt
```

- 마지막 경로 구성 요소를 가리킴(문자열)
 - 디렉터리일 수도, 일반 파일일 수도 있음
- 단일 항목을 추출해야 할 경우에 유용



```
from pathlib import Path

p = Path("file.txt")

print(p.suffix)
# .txt

p = Path.cwd()
print(p.suffix)
#

# 이름이 "dir.dir"인 디렉터리
p = Path.cwd() / "dir.dir"
print(p.suffix)
# .dir
```

- 파일의 확장자를 가리킴(문자열)
- 확장자가 없는 경우에는 빈 문자열을 가짐
- 단순한 어휘 분석이므로 요소의 마지막 '.'부터의 내용을 담고 있음
 - 디렉터리에도 동일하게 동작하므로 `is_file` 등의 메서드로 검증 필요

```
from pathlib import Path

p = Path("file.txt")

print(p.stem)
# file

# C:\Users\Name\dir
p = Path.cwd()
print(p.stem)
# dir

# 이름이 "dir.dir"인 디렉터리
p = Path.cwd() / "dir.dir"
print(p.stem)
# dir
```

- 파일의 확장자를 제외한 이름을 가리킴(문자열)
- 확장자가 없는 경우 이름과 동일
- 단순한 어휘 분석이므로 디렉터리에 '.'가 있어도 앞부분만을 가리킴
 - `is_dir` 등의 메서드로 검증 필요



```
import os
from pathlib import Path

p = Path.cwd()

print(p)
# C:\Users\Name\dir

os.chdir("..")

p = Path.cwd()

print(p)
# C:\Users\Name
```

- 작업 디렉터리를 변경
- 문자열 이외에 Path 객체의 전달도 가능
- pathlib이 아닌 os 라이브러리에서 지원하는 기능
 - "import os"를 통해 사용

4. 파일 변경하기



- **replace**: 경로가 가리키는 파일 혹은 디렉터리 교체 및 교체된 경로 반환
- **touch**: 경로에 빈 파일 만들기
- **unlink**: 경로가 가리키는 일반 파일 제거
- **mkdir**: 경로에 디렉터리 생성
- **rmdir**: 경로가 가리키는 빈 디렉터리 제거

```
from pathlib import Path

# 이름 바꾸기
p = Path("dir/oldname")
p_new = p.parent / "newname"
# "oldname"이 존재하는 디렉터리에서 이름 변경
if not p_new.exists():
    p = p.replace(p_new)
    print(p)
    # dir\newname

# 위치 바꾸기
p = Path("testfile.txt")
p_new = Path("dir/" + p.name)
# 파일을 작업 디렉터리에서 "dir" 디렉터리로 이동
if not p_new.exists():
    p = p.replace(p_new)
    print(p)
    # dir\testfile.txt
```

- Path에 해당하는 파일 경로 및 이름을 다른 Path로 교체
 - Path 객체나 문자열 전달 가능
- 절대 경로를 전달하거나 작업 디렉터리 기준으로 상대 경로를 전달해야 함에 유의
 - Path에 해당하는 경로 기준이 아님
- 변경하려는 경로에 이미 파일이 존재하는 경우 교체됨
 - 단순히 이동만 하려면 항상 **exists** 메서드를 사용하여 확인하는 것이 좋음

```
from pathlib import Path

p = Path("new_file.txt")

print(p.exists())
# False
p.touch()
print(p.exists())
# True

p = Path("existing_file.txt")

print(p.exists())
# True
p.touch(exist_ok=False)
# FileNotFoundError
```

- 경로에 빈 파일을 생성
- 만약 파일이 이미 존재하는 경우 수정 시각을 지금으로 바꿈
 - `exist_ok` 매개 변수에 `False`를 전달하면 `FileExistsError` 발생

```
from pathlib import Path

p = Path("existing_file.txt")

print(p.exists())
# True
p.unlink()
print(p.exists())
# False

p.unlink(missing_ok=True) # no error

p = Path()

p.unlink()
# OSError
```

- 경로에 해당하는 디렉터리가 아닌 파일을 제거
- 경로에 해당 파일이 존재하지 않으면 **FileNotFoundError** 발생
 - **missing_ok**에 True를 전달하면 존재 유무와 상관 없이 오류가 발생하지 않음
- 디렉터리인 경우 **OSError** 하위의 예외 발생(운영 체제별로 다름)


```
from pathlib import Path

# C:\Users\Name\dir
p = Path.cwd()
# 아직 실존하지 않는 경로
# C:\Users\Name\dir\dir_example
p /= "dir_example"

print(p.exists())
# False

p.mkdir()

print(p.exists())
# True
```

- 디렉터리 생성
- Path 객체로서 단순히 논리적으로 정의된 경로를 파일 시스템 상의 실제 경로로 만드는 작업



```
from pathlib import Path

# C:\Users\Name\dir
p = Path.cwd()
# 아직 어떠한 디렉터리도 존재하지 않음
p /= "dir1/dir2/dir3"

p.mkdir(parents=True) # okay
for parent in p.parents:
    print(parent)
# C:\Users\Name\dir\dir1\dir2
# C:\Users\Name\dir\dir1
# C:\Users\Name\dir
# C:\Users\Name
```

- **parents**: 상위 디렉터리가 없을 경우 어떻게 처리할지 결정(기본값: False)
- True로 설정하면 지정한 디렉터리까지 거쳐야 하는 디렉터리를 전부 생성
- False로 설정하면 상위 디렉터리가 존재하지 않으면 **FileNotFoundError** 발생



```
from pathlib import Path

p = Path()
p.mkdir(exist_ok=True)  # okay

p = Path("existing_file.txt")
p.mkdir(exist_ok=True)
# FileNotFoundError
```

- **exist_ok**: **mkdir**을 호출하는 시점에 경로가 이미 존재할 때 어떻게 처리할지 결정(기본값: False)
- True로 설정하면 해당 경로가 디렉터리로 이미 존재할 때 아무 동작도 수행하지 않음
- False로 설정하면 위의 경우 **FileExistsError** 발생
- 이미 존재하는 경로가 디렉터리가 아닌 경우 **exist_ok**의 여부와 관계 없이 **FileExistsError** 발생

```
from pathlib import Path

p = Path("empty_directory")

print(p.exists())
# True
p.rmdir()
print(p.exists())
# False
```

- 경로가 빈 디렉터리일 때 해당 디렉터리 제거
- 경로가 비어 있지 않은 경우 **OSError** 발생
- 실제 존재하지 않는 경로인 경우 **FileNotFoundError** 발생

- 파일에 대한 고수준 연산을 제공하는 라이브러리
- pathlib에 존재하지 않는 파일 복사와 삭제를 할 수 있음
- **copyfile**
 - 파일 복사, 만약 복사하려는 파일 이름이 이미 존재하는 경우 교체됨
- **copytree**
 - 디렉터리 복사, 디렉터리 안에 있는 내용을 같이 복사함
- **rmtree**
 - 디렉터리 제거, pathlib의 **Path.rmdir**과 달리 디렉터리 안에 파일이 존재해도 같이 삭제함

```
from pathlib import Path
from shutil import copyfile

p = Path("source_file.txt")
file = copyfile(p, "dir/" + p.name)

print(file)
# dir/source_file.txt
```

- **copyfile(원본, 사본)** 형식으로 사용
 - 문자열 혹은 바이트열로 변환 가능한 값을 인자로 받음
 - Path 객체는 자동으로 문자열로 변환됨
 - 사본의 위치를 문자열 혹은 바이트열로 반환
- 사본을 저장하려는 파일의 이름까지 적어야 함
 - 디렉터리까지만 작성하는 경우 **OSError** 하위의 예외 발생
- 사본으로 정하려는 이름이 이미 존재하는 경우 교체됨

```
from pathlib import Path
from shutil import copytree

p = Path("dir")
p_copy = copytree(p, "dir_copy")

for root, dirs, files in Path(p_copy).walk():
    print(root, dirs, files)
    # dir_copy ['dir1'] ['file1.txt', 'file2.txt']
    # dir_copy\dir1 ['dir2'] ['file3.txt', 'file4.txt']
    # dir_copy\dir1\dir2 [] ['file5.txt', 'file6.txt']
```

- 디렉터리 안의 내용도 같이 복사함
- **copytree(원본, 사본)** 형식으로 사용
- **dirs_exist_ok** 매개 변수에 따라 이미 사본이 있는 경우에 대한 동작이 다름(기본값: False)
 - False인 경우 **FileExistsError** 발생
 - True인 경우 사본에 있는 파일 중 원본과 이름이 같은 파일을 모두 원본으로 교체

```
from pathlib import Path
from shutil import rmtree

p = Path("dir")

for root, dirs, files in p.walk():
    print(root, dirs, files)
    # dir_copy ['dir1'] ['file1.txt', 'file2.txt']
    # dir_copy\dir1 ['dir2'] ['file3.txt', 'file4.txt']
    # dir_copy\dir1\dir2 [] ['file5.txt', 'file6.txt']

rmtree(p)
print(p.exists())
# False
```

- 디렉터리의 내부 파일 여부에 상관없이 삭제
- 디렉터리 안의 파일이 삭제할 수 없는 상태일 때 **ignore_errors** 매개 변수를 통해 진행 여부 결정(기본값: False)
 - True인 경우 삭제할 수 없는 파일은 두고 나머지 파일 삭제
 - False인 경우 작업 중단